

MQTT for IoT Systems

The Problem with REST API Alone

Our REST API system works — but IoT devices have special needs.

REST API Model (Request/Response):

Device ——— POST /api/data ———> Server
Device <—— 200 OK ——— Server

OK Good for: sending data once in a while

✗ Problem: what if data changes every second?

Imagine a temperature sensor sending every second:

Device sends 1 reading/sec × 1000 devices = 1000 HTTP requests/sec

Each HTTP request:

- Opens TCP connection
 - Sends headers (300–800 bytes overhead)
 - Waits for response
 - Closes connection
- Too heavy for small devices!
- Too many connections for the server!

HTTP was designed for web pages, not high-frequency sensor streams.

What is MQTT?

MQTT = Message Queuing Telemetry Transport

A **lightweight messaging protocol** designed specifically for IoT:

- Created in 1999 for oil pipeline monitoring (low bandwidth, unreliable links)
- Uses **Publish/Subscribe** pattern instead of Request/Response
- Very small message overhead (~2 bytes minimum header)
- Runs over TCP — persistent connection, no reconnect overhead

Publish/Subscribe Pattern

REST (Request/Response):

Device → Server (device must know server address)

MQTT (Publish/Subscribe):

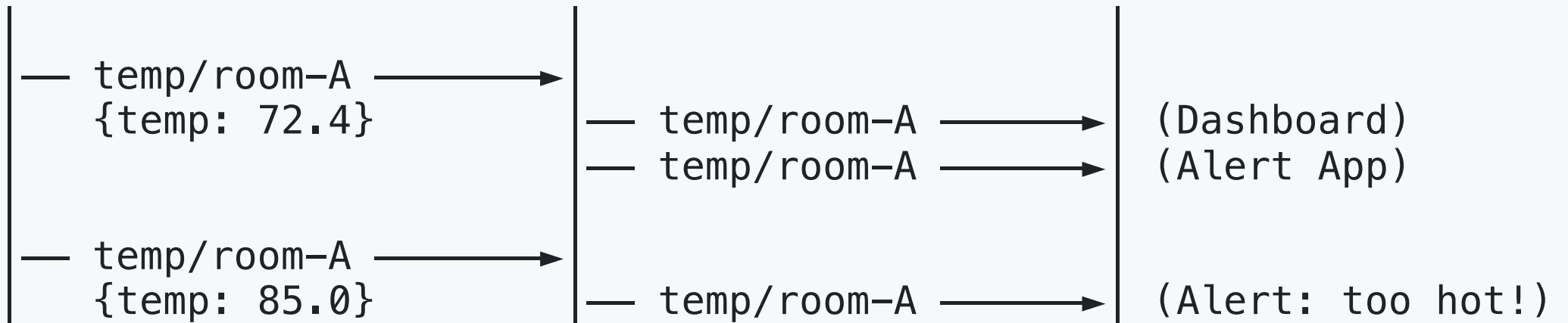
Device → Broker → All Subscribers
(central hub)

The device **publishes** to a topic. Anyone **subscribed** to that topic receives it automatically — without polling.

Publisher
(ESP32 Sensor)

Broker
(MQTT Server)

Subscribers
(Dashboard / Alert)



One publisher, many subscribers — no direct connection between them.

MQTT Topics

Topics are **hierarchical paths** (like file system paths):

```
sensors/building-A/room-1/temperature  
sensors/building-A/room-1/humidity  
sensors/building-B/room-3/temperature  
actuators/building-A/hvac/setpoint
```

Topic Wildcards

Single level wildcard (+):

sensors/+/room-1/temperature

→ matches: sensors/building-A/room-1/temperature

→ matches: sensors/building-B/room-1/temperature

Multi level wildcard (#):

sensors/#

→ matches: sensors/building-A/room-1/temperature

→ matches: sensors/building-A/room-2/humidity

→ matches: everything under sensors/

Wildcards let dashboards subscribe to all sensors at once.

Quality of Service (QoS)

MQTT has 3 delivery guarantees:

QoS 0 – "Fire and forget"

Device sends once. No confirmation. May be lost.

→ Use for: frequent readings where occasional loss is OK

QoS 1 – "At least once"

Broker confirms receipt. May deliver duplicates.

→ Use for: important data, device can handle duplicates

QoS 2 – "Exactly once"

Four-way handshake. Guaranteed exactly once.

→ Use for: actuator commands, billing data

How to Apply QoS

Publisher Side

When publishing, specify QoS level.

Example:

```
client.publish("sensor/temp", value, qos=1)
```

Subscriber Side

When subscribing, you also request a QoS:

```
client.subscribe("sensor/temp", qos=1)
```

Important rule:

```
Final QoS = MIN(Publisher QoS, Subscriber QoS)
```

Example:

Publisher = QoS 2 + Subscriber = QoS 1 → Delivered as QoS 1

The MQTT Broker

The **broker** is the central hub — it receives all published messages and routes them to subscribers.

MQTT Broker
(Mosquitto / EMQX / HiveMQ)

Topics:

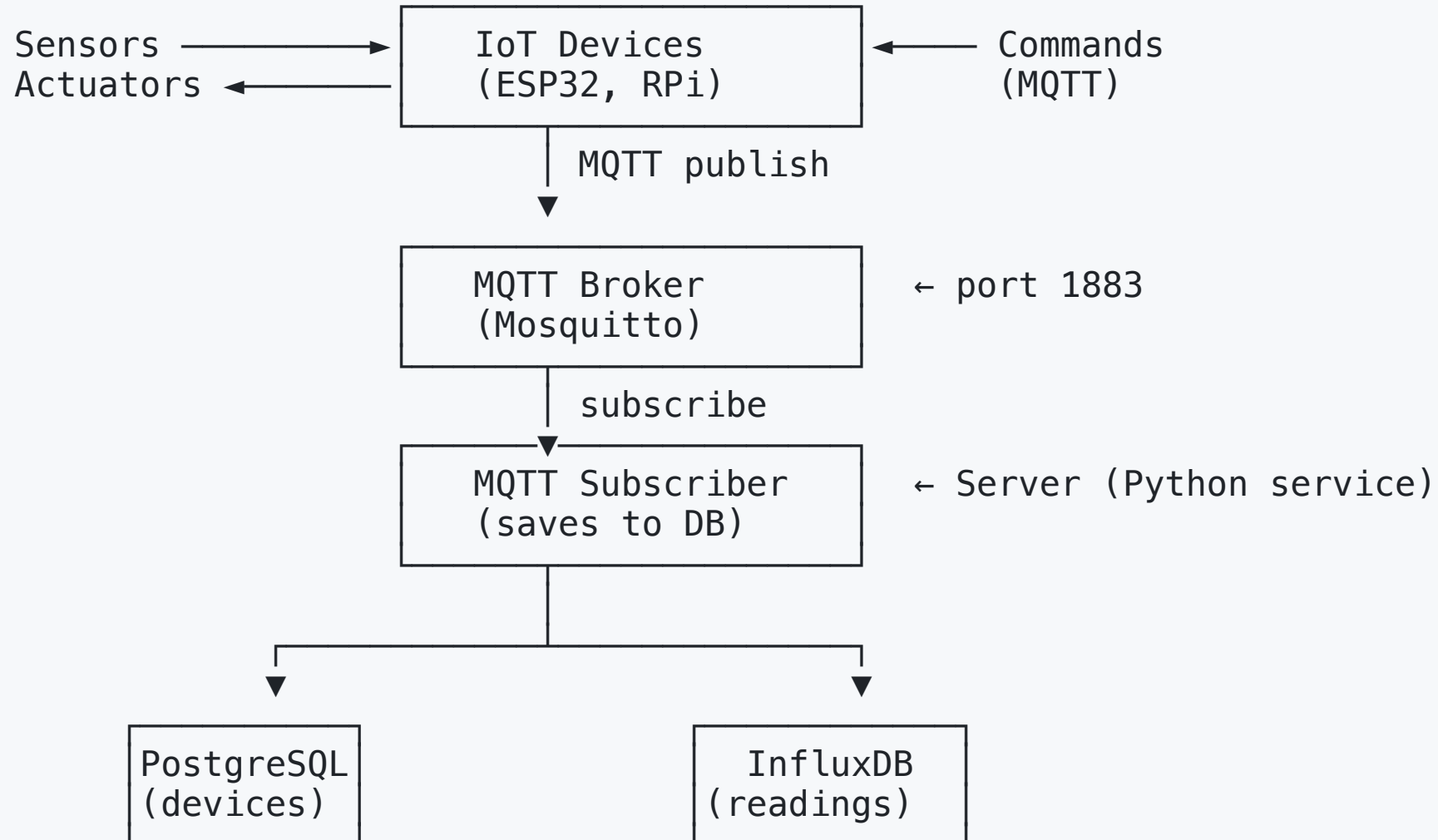
sensors/room-A/temp	→ [sub1, sub2]
sensors/room-B/temp	→ [sub1]
actuators/hvac/#	→ [sub3]

Mosquitto is the most popular open-source MQTT broker —

REST vs MQTT — When to Use Which

Situation	Use REST	Use MQTT
Register a new device	OK	
Send sensor data every second		OK
User logs in from dashboard	OK	
Push alert to 1000 devices		OK
Query historical data	OK	
Real-time live sensor feed		OK
Firmware update metadata	OK	
Send actuator command		OK

Complete Architecture with MQTT



Commands (MQTT)

In this example, we assume that the IoT Devices (ESP32, Rpi) also subscribe to command topics for real-time control:

- For example, the dashboard can publish to `actuators/room-A/hvac` to set the HVAC temperature (`mqtt.subscribe("actuators/room-A/hvac");`).
- Then, the MQTT Broker routes that command to the appropriate device subscribed to that topic.
- This allows for **bidirectional communication** — not just sending data, but also receiving commands.

MQTT + REST Together

Device Registration (once):

ESP32 — REST POST /api/devices —> Flask API —> PostgreSQL

Sensor Streaming (every second):

ESP32 — MQTT publish —> Mosquitto —> Python subscriber —> InfluxDB

Dashboard Query (on demand):

Browser — REST GET /api/readings —> Flask API —> InfluxDB

Actuator Command (real-time):

Dashboard — REST POST /api/commands —> Flask API —> MQTT publish —> ESP32

REST for management and queries. MQTT for real-time data streams and commands.

MQTT Security — Authentication

By default, Mosquitto accepts any connection. For production, enable authentication:

```
# Create password file
mosquitto_passwd -c /etc/mosquitto/passwd esp32-device

# mosquitto.conf
listener 1883
allow_anonymous false
password_file /etc/mosquitto/passwd
```

```
// ESP32: connect with credentials
mqtt.connect("esp32-001", "esp32-device", "secret-password");
```